

Sprawozdanie Projekt 2 OE:

Szymon Poznański

Hubert Chruścicki

Przemysław Wojnar

Technologie użyte w projekcie.

Aplikacja została zaprojektowana w architekturze klient-serwer jako nowoczesna aplikacja webowa Full-Stack. Podział responsywności i logiki biznesowej zrealizowano przy pomocy następujących technologii:

Część serwerowa - Backend:

- **Python** – Główny język programowania wykorzystany na serwerze. Idealnie nadaje się do implementacji złożonych obliczeń matematycznych, takich jak logika algorytmów ewolucyjnych.
- **Flask** – Lekki i elastyczny framework webowy języka Python. Posłużył on do stworzenia serwera oraz zbudowania interfejsu REST API, który obsługuje żądania napływające z aplikacji klienckiej i zwraca wyniki optymalizacji.
- **Swagger UI / OpenAPI** – Narzędzie zintegrowane w ścieżce /api/docs, służące do interaktywnego podglądu, testowania i dokumentacji wystawionych punktów końcowych aplikacji backendowej.
- **Jupyter Notebook** – Wykorzystany dodatkowo do szybkiego prototypowania, weryfikacji i testowania działania samego algorytmu analitycznego w izolacji od serwera.

Część kliencka - Frontend:

- **React** – Popularna biblioteka języka JavaScript służąca do budowy interaktywnych interfejsów użytkownika. Aplikacja frontendowa dzieli widoki na wielokrotnego użytku komponenty m.in. obsługujące formularze parametryzacji algorytmu oraz sekcje prezentacji wyników optymalizacji.
- **TypeScript** – Statycznie typowany nadzbiór języka JavaScript. Zapewnia on silne typowanie danych w strukturze projektu frontendowego, co minimalizuje ilość błędów w czasie pisania kodu i znacznie poprawia jego czytelność w szerszych aplikacjach.
- **Vite** – Nowoczesne i bardzo wydajne narzędzie budujące nowej generacji dla aplikacji webowych. Gwarantuje znacznie szybszy czas uruchamiania środowiska deweloperskiego i kompilacji w porównaniu do standardowych narzędzi takich jak Webpack.
- **HTML & CSS** – Standardowe technologie wykorzystane do definicji struktury i kaskadowego stylowania widoków na stronie.

Wymagania środowiskowe

Do poprawnego uruchomienia obu warstw aplikacji (serwerowej i klienckiej) oraz pełnego wykorzystania jej funkcjonalności, środowisko uruchomieniowe musi spełniać poniższe wymagania:

Środowisko dla części backendowej

- **Zainstalowany interpreter języka Python** - Wersja 3.8 lub nowsza.
- **Menedżer pakietów PIP** - Narzędzie do instalacji wymaganych zewnętrznych pakietów dla Pythona.
- **Wirtualne środowisko** - Zaleca się uruchomienie serwera w odizolowanym środowisku (np. moduł venv), aby uniknąć konfliktów zależności z systemem operacyjnym.
- **Biblioteki zależności:** Wszystkie pakiety wymagane do działania serwera zdefiniowane są w pliku requirements.txt.

Środowisko dla części frontendowej

- **Środowisko Node.js** - Wersja 18.x lub nowsza, wymagana do działania narzędzi budujących Vite oraz uruchomienia serwera deweloperskiego.
- **Menedżer pakietów NPM** - Wymagany do pobrania lokalnych pakietów zdefiniowanych w pliku package.json, m.in. biblioteki React, Material-UI, Recharts.

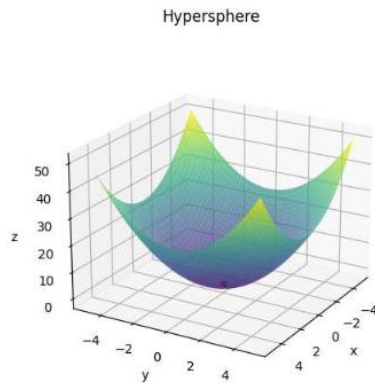
Oprogramowanie dodatkowe użytkownika

- **Przeglądarka internetowa** - Nowoczesna przeglądarka ze wsparciem dla standardu ES6 do wyświetlania interfejsu graficznego.
- **Edytor kodu** - Opcjonalnie, do modyfikacji i przeglądania kodu, posiadający zintegrowany terminal w celu łatwego zarządzania procesami.

Opis wybranych funkcji testowych i ich optima.

Silnik aplikacji został zaprojektowany w sposób obiektowy, wykorzystując interfejsy i klasy abstrakcyjne, co pozwala na łatwe dodawanie nowych wariantów funkcji matematycznych. Obecnie w procesie ewaluacji algorytmu genetycznego wykorzystywana jest następująca wielowymiarowa funkcja testowa:

Funkcja Sferyczna (Hypersphere) - jest to jedna z najbardziej klasycznych, ciągłych i wypukłych funkcji testowych, powszechnie stosowana jako punkt odniesienia sprawdzający stopień zbieżności powoływanych do życia algorytmów optymalizacyjnych. Jej wykres przypomina wielowymiarową parabolę.



- **Wzór ogólny:** wartość dostosowania obliczana jest jako suma kwadratów wszystkich zmiennych decyzyjnych danego osobnika, co można opisać wzorem:

$$f(x) = \sum_{i=1}^N x_i^2$$

- **Przyjęta dziedzina poszukiwań:** każda zmienna testowa jest standardowo inicjowana w przedziale od -5.0 do 5.0.
- **Optimum - minimum globalne:** zadanie algorytmu opiera się na problemie minimalizacji. Oczekiwane, idealne i absolutne optimum globalne - najniższa możliwa wartość funkcji wynosi dokładnie 0.0 jako wartość samej funkcji (fitness), a odnajdywane jest tylko w jednym punkcie środkowym układu współrzędnych, dla wektora decyzji w którym wszystkie składowe przyjmują zerowe wartości precyzyjne: $x = [0.0, 0.0, \dots, 0.0]$

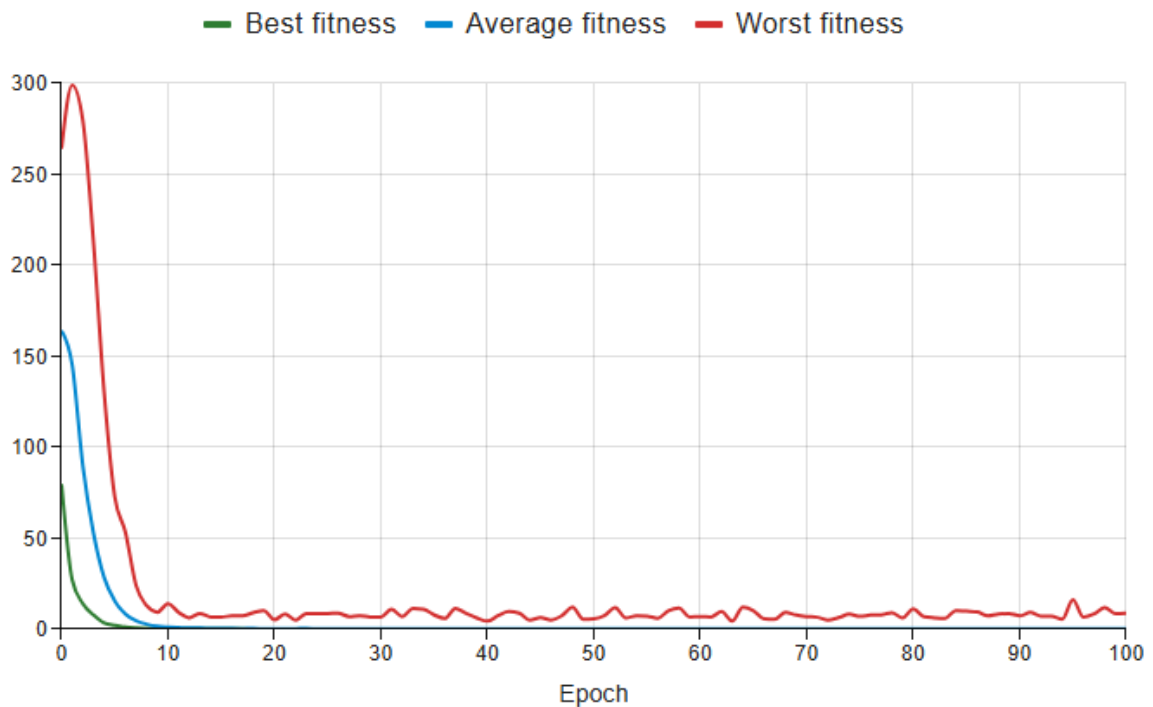
Z racji precyzji obliczeń numerycznych, dobrym i oczekiwanym rezultatem pracy algorytmu ewolucyjnego jest moment, w którym populacja we wszystkich wymiarach zbliża się możliwie najbliższej wartości idealnej wynoszącej 0.0.

Wykresy wyników

Wykres 1 (Konfiguracja Zbalansowana)

Ustawienia :

- Liczba zmiennych: 20
- Wielkość populacji: 500
- Liczba epok: 100
- Metoda selekcji: Tournament
- Metoda krzyżowania: Linear (Prawdopodobieństwo: 0.8)
- Metoda mutacji: Gaussian (Prawdopodobieństwo: 0.01 Odchylenie sigma 0.1)
- Strategia elitarna: Włączona



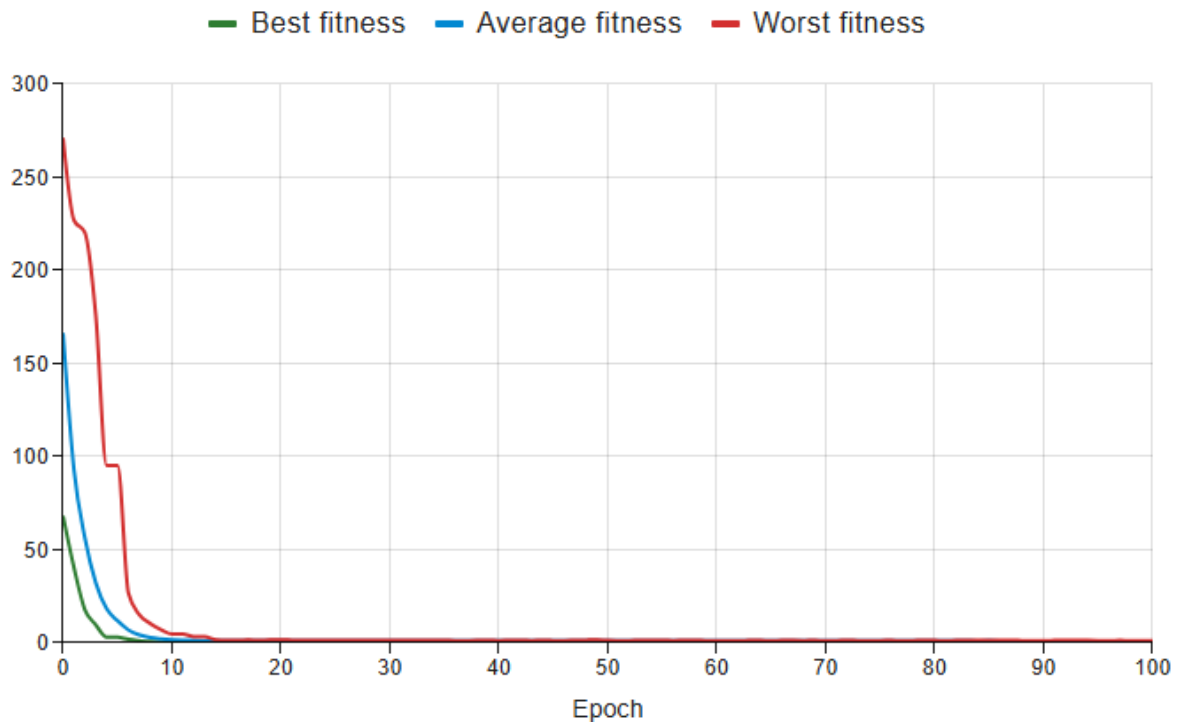
Wnioski:

- **Szybka zbieżność:** Algorytm błyskawicznie znajduje optymalne rozwiązania, co widać po drastycznym spadku zielonej linii niemal do zera. To zasługa selekcji turniejowej i krzyżowania linearnego, które sprawnie ściągają populację do minimum.
- **Zgodność populacji:** Średnia ocena wszystkich osobników opada tuż za najlepszym z nich. Oznacza to, że populacja uczy się na bieżąco i większość genów bezpiecznie skupia się wokół dobrych rozwiązań.
- **Zachowana różnorodność:** Najgorsze osobniki stale skaczą na wykresie i mają słabe wyniki. Wynika to z działania mutacji Gaussa, która delikatnie zaburza geny, dzięki temu algorytm zachowuje różnorodność i nie utyka w martwym punkcie.
- **Stabilność najlepszych:** Wynik najlepszego osobnika nigdy się nie pogarsza po dotarciu do dna. Jest to bezpośredni dowód na to, że włączona strategia elitarna idealnie działa i skutecznie chroni wypracowane już rozwiązania przed zniszczeniem.

Wykres 2 (Agresywna Eksploatacja)

Ustawienia :

- Liczba zmiennych: 20
- Wielkość populacji: 500
- Liczba epok: 100
- Metoda selekcji: Best Selection
- Metoda krzyżowania: Averaging (Prawdopodobieństwo: 0.85)
- Metoda mutacji: Gaussian (Prawdopodobieństwo: 0.01 Odchylenie sigma 0.01)
- Strategia elitarna: Włączona



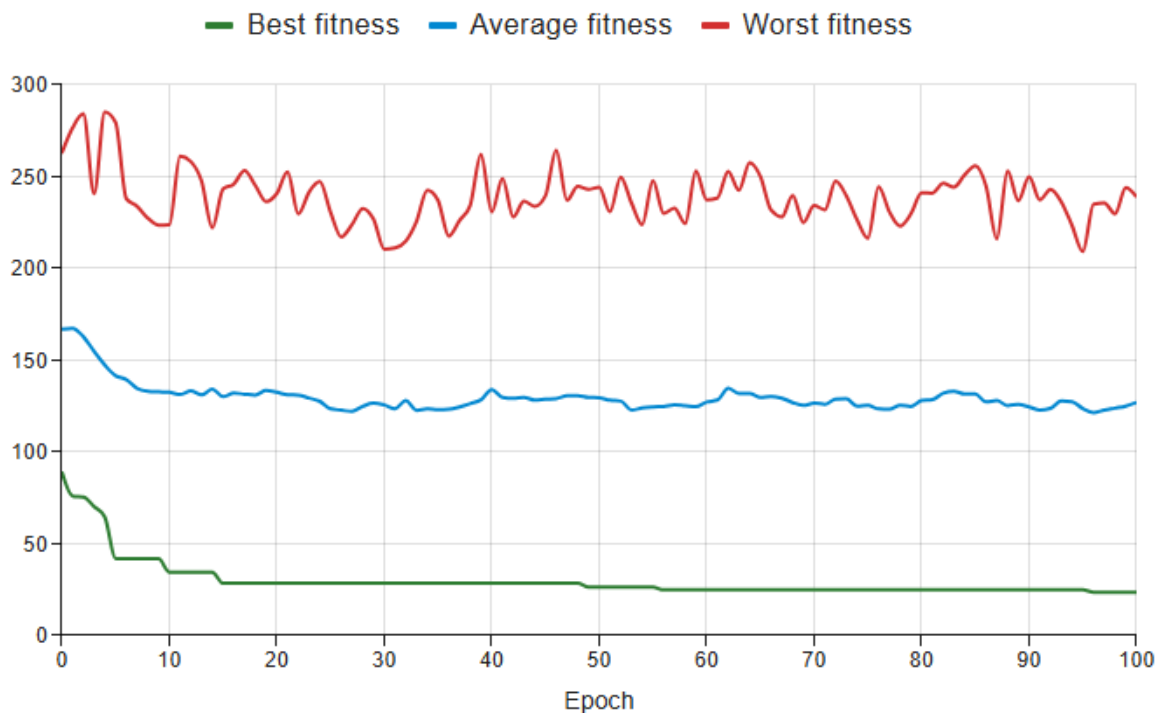
Wnioski:

- **Błyskawiczna stagnacja:** Algorytm niemal natychmiast utknął w miejscu, co widać po wczesnym wypłaszczeniu się krzywych na wykresie. To bezpośredni skutek wyboru selekcji Best Selection, która dopuszcza do rozrodu tylko wąską elitę.
- **Utrata różnorodności:** Krzyżowanie uśredniające drastycznie i błyskawicznie ujednoliciło geny całej populacji. W rezultacie średnie i najgorsze wyniki bardzo szybko zlały się z linią najlepszego osobnika.
- **Zbyt słaba mutacja:** Mutacja Gaussa z mikroskopijnym odchyleniem sigma była zbyt delikatna, aby uratować różnorodność. Nie potrafiła wygenerować na tyle silnego zaburzenia, aby wybić algorytm z pułapki lokalnego minimum.
- **Przedwczesna zbieżność:** Algorytm postawił na ekstremalną eksploatację pierwszych lepszych rozwiązań, całkowicie porzucając eksplorację dziedziny. Z tego powodu optymalizacja zakończyła się zbyt wcześnie na słabym wyniku.

Wykres 3 (Maksymalna Eksploatacja)

Ustawienia :

- Liczba zmiennych: 20
- Wielkość populacji: 500
- Liczba epok: 100
- Metoda selekcji: Roulette
- Metoda krzyżowania: Blend (alfa-beta) (Prawdopodobieństwo: 0.9, alfa 0.5, beta 0.5)
- Metoda mutacji: Uniform (Prawdopodobieństwo: 0.15)
- Strategia elitarna: Włączona



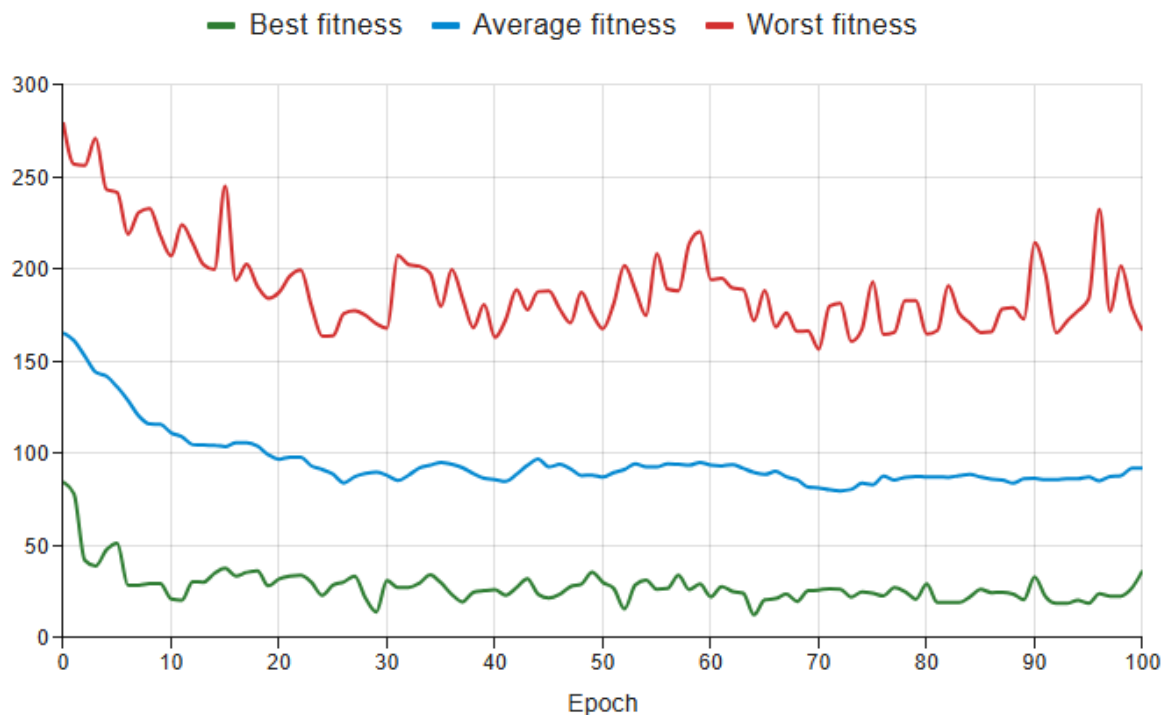
Wnioski:

- **Ekstremalny chaos:** Algorytm działa niemal jak losowe błędzenie po dziedzinie. Bardzo silna mutacja równomierna (15%) ciągle "rozrzuca" osobniki, co widać po wysokiej i mocno poszarpanej czerwonej linii.
- **Brak precyzji:** Populacja nie potrafi skupić się na dopracowywaniu rozwiązań. Szerokie krzyżowanie (alfa-beta) oraz słaba selekcja ruletkowa sprawiają, że średni wynik (niebieska linia) w ogóle nie spada i utrzymuje się bardzo wysoko.
- **Ratunek w elicie:** Najlepszy wynik (zielona linia) poprawia się niezwykle powoli i nie osiąga zera. Utrzymuje on swój poziom wyłącznie dzięki włączonej strategii elitarnej, która chroni go przed zniszczeniem przez agresywne operatory.

Wykres 4 (Test bez zabezpieczeń)

Ustawienia :

- Liczba zmiennych: 20
- Wielkość populacji: 500
- Liczba epok: 100
- Metoda selekcji: Roulette
- Metoda krzyżowania: Blend (alfa) (Prawdopodobieństwo: 0.7, alfa 0.5)
- Metoda mutacji: Uniform (Prawdopodobieństwo: 0.05)
- Strategia elitarna: Wyłączona



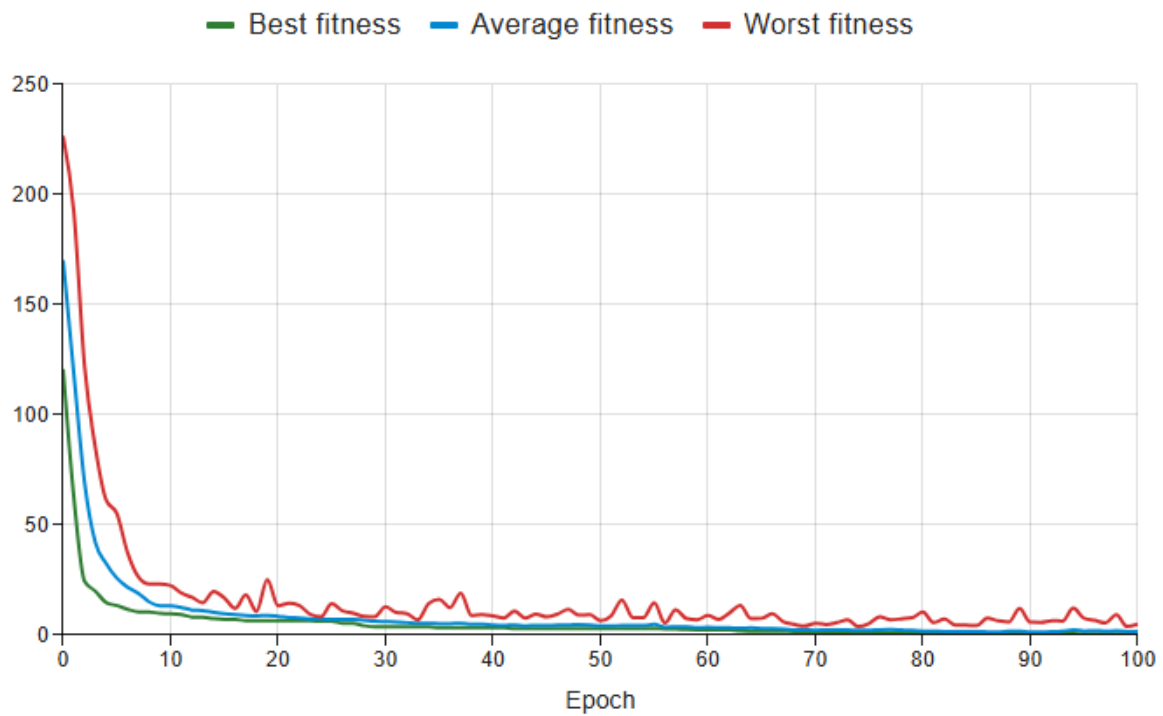
Wnioski:

- **Zjawisko regresji:** Brak strategii elitarniej sprawia, że zielona linia najlepszego wyniku często "skacze" w górę zamiast tylko opadać. Oznacza to, że algorytm na bieżąco niszczy i zapomina wypracowane już, dobre rozwiązania.
- **Ciągła niestabilność:** Krzyżowanie rozszerzające (alfa) i mutacja równomierna wprowadzają do systemu szum, z którym słaba selekcja ruletkowa zupełnie sobie nie radzi. W efekcie cała populacja (niebieska linia) nie potrafi płynnie i trwale zbiegać do optymalnego minimum.
- **Kluczowa rola elity:** Nawet gdy algorytm przypadkiem znajdzie się bliżej zera, w kolejnej epoce ten wynik bezpowrotnie przepada. To ostatecznie udowadnia, że fizyczna ochrona najlepszych osobników jest w tym problemie absolutnie niezbędna do osiągnięcia sukcesu.

Wykres 5 (Mała populacja)

Ustawienia :

- Liczba zmiennych: 20
- Wielkość populacji: 500
- Liczba epok: 100
- Metoda selekcji: Roulette
- Metoda krzyżowania: Arithmetic (Prawdopodobieństwo: 0.8, alfa 0.5)
- Metoda mutacji: Gaussian (Prawdopodobieństwo: 0.02, sigma 0.1)
- Strategia elitarna: Włączona



Wnioski:

- **Błyskawiczna utrata różnorodności:** Zbyt mała pula początkowa drastycznie ogranicza dostępny materiał genetyczny. Widać to na wykresie, gdzie wszystkie trzy linie bardzo szybko zbliżają się do siebie i ujednolicają.
- **Przedwczesna stagnacja:** Krzyżowanie arytmetyczne szybko uśrednia i tak już nieliczną populację. Algorytm przedwcześnie "utyka", a wykres wygładza się na dość słabym wyniku (daleko od zera).
- **Brak siły napędowej:** Delikatna mutacja Gaussa nie potrafi wygenerować wystarczającego szumu, aby uratować tak małą populację. Algorytmowi brakuje po prostu unikalnych osobników, aby skutecznie przeszukać trudną, 20-wymiarową przestrzeń.

Porównanie wyników dla różnych konfiguracji algorytmu: chromosom **binarny** vs **realny**

Zestawienie wyników optymalizacji

Numer wykresu	Konfiguracja	Najlepszy wynik (binarny) (Best Fitness)	Najlepszy wynik (realny) (Best Fitness)	Średni wynik populacji (binarny)	Średni wynik populacji (realny)	Czas wykonania (binarny) (s)	Czas wykonania (realny) (s)
1	Konfiguracja Zbalansowana	0.0015	0.0026	0.66	0.22039	4.55	1.4004
2	Agresywna Eksploatacja (Best Selection)	19.29	0.5058	150.72	1.0323	3.54	0.7457
3	Maksymalna Eksploracja (Wysoka mutacja)	8.26	23.6365	50.82	127.0588	7.12	1.0744
4	Test bez zabezpieczeń (Brak Elitaryzmu)	50.11	36.5429	193.53	92.1593	4.23	0.9221
5	Mała populacja (40 osobników)	7.46	0.6536	11.76	1.663	0.34	0.0649

Analiza porównawcza i wnioski:

- **Bezkonkurencyjna wydajność czasowa:** We wszystkich badanych scenariuszach reprezentacja rzeczywista drastycznie przyspieszyła proces optymalizacji. Brak konieczności ciągłego rzutowania i dekodowania łańcuchów bitowych zdejmując z silnika algorytmu ogromny narzut obliczeniowy.
- **Lepsza jakość całej populacji:** Chromosom realny nie tylko znajdował lepsze rozwiązania na czele populacji, ale równocześnie podciągał wyniki całej reszty osobników. W reprezentacji binarnej często dochodziło do sytuacji, gdzie jeden świetny osobnik istniał w otoczeniu bardzo słabych - większość populacji nie nadążała za elitą i nie uczyła się od niej.
- **Większa odporność na przedwczesną zbieżność:** Tam gdzie agresywna selekcja i uśredniające krzyżowanie blokowały algorytm daleko od minimum, chromosom realny wciąż znajdował drogę naprzód. Reprezentacja binarna w tych samych warunkach utykała znacznie wcześniej i na znacznie gorszym wyniku.
- **Binarna przewaga w warunkach wysokiego szumu:** Przy silnej mutacji i słabej selekcji chromosom binarny potrafił zachować lepszego najlepszego osobnika niż jego realny odpowiednik. Dyskretna natura reprezentacji binarnej działa w chaotycznych warunkach jak naturalny filtr, chroniąc elitę przed całkowitym rozpadem.
- **Reprezentacja realna jako bezpieczniejszy wybór ogólny:** Zestawiając wszystkie konfiguracje, chromosom realny okazał się rozwiązaniem bardziej uniwersalnym - szybszym, stabilniejszym populacyjnie i mniej wrażliwym na dobór parametrów.

Chromosom binarny może mieć rację bytu w specyficznych scenariuszach eksploracyjnych, jednak wymaga znacznie staranniejszego strojenia operatorów, aby dorównać jakości.